

FESTOOL

GEMEINSAM AUSPROBIEREN

Mit KI selbst bauen

Festool KI-Hackathon
15. September 2026, Wendlingen



Manuel Fuß

Unterlagen & Vorlagen

manuel-fuss.de/festool



Heute entsteht etwas Eigenes

Erst ein Spiel.

Dann eure Aufgaben.

Jeder probiert selbst aus.

Wir zeigen zwischendurch, was schon funktioniert.

Was bei euch daraus werden kann

Ausgangsmaterial	Eine mögliche erste Version
Meeting-Notizen	Ein Helfer, der User Stories vorschlägt.
Video-Transkript	Eine verständliche Anleitung mit einzelnen Schritten.
Reporting-Export	Ein Tool, das fehlende Werte und Abweichungen zeigt.

Am Anfang reicht ein klarer Ausschnitt einer echten Aufgabe.

KI hilft auch beim Bauen selbst

Ihr kennt die Aufgabe

Was soll leichter werden?

Woran erkennt ihr ein gutes Ergebnis?

Was darf das Tool tun?

Die KI arbeitet daran mit

Sie erklärt unbekannte Begriffe.

Sie schreibt und ändert
Projektdateien.

Sie hilft, Fehler einzugrenzen.

Ein Screenshot kann der KI zeigen, wo ihr gerade festhängt.

Unsere Werkzeuge heute

Claude Code

Für eigene kleine Anwendungen.

Arbeitet mit Dateien im Projektordner.

Wir testen das Ergebnis im Browser.

Die Spielübung zeigt den Einstieg mit Claude Code.

Copilot Studio

Für Assistenten und Arbeitsabläufe.

Verbindet Schritte über eine Oberfläche.

Anbindungen hängen von euren Freigaben ab.

Ein Projekt ist erst einmal ein Ordner

mein-spiel/

src/

Hier liegt der Anwendungscode.

package.json

Hier stehen Pakete und Startbefehle.

README.md

Hier steht, wie das Projekt funktioniert.

Ein eigener Ordner pro Idee hilft euch und der KI, den Überblick zu behalten.

Browser und Terminal

Im Browser

Ihr seht eure Anwendung.

Ihr klickt, spielt und prüft das Ergebnis.

Im Terminal

Ihr gebt Befehle als Text ein.

Ihr startet Werkzeuge und seht Meldungen.

Ein Befehl wie „cd mein-spiel“ wechselt in euren Projektordner.

Was der Browser versteht

Begriff	Am Beispiel eines Spiels
HTML	Die Struktur: Überschrift, Spielfeld, Startknopf.
CSS	Das Aussehen: Farben, Abstände und Schrift.
JavaScript	Das Verhalten: Ein Klick startet die Runde.

Die KI kann diese Dateien schreiben. Ihr beschreibt das gewünschte Verhalten.

React

Bausteine für die Oberfläche

React ist eine JavaScript-Bibliothek.
Sie hilft, interaktive Oberflächen zu bauen.

In unserem Spiel: Startknopf, Punktestand und Spielfeld.

Wenn sich der Punktestand ändert, aktualisiert React die Anzeige.

Node.js

JavaScript außerhalb des Browsers

Node.js führt JavaScript auf einem Rechner aus.

Heute laufen damit unsere Entwicklungswerkzeuge
und der lokale Vorschau-Server.

Die Oberfläche sehen wir weiterhin im Browser.

npm und Vite

npm

Verwaltet fertige Code-Pakete.

Installiert, was das Projekt braucht.

Startet hinterlegte Projektbefehle.

Vite

Legt das Grundgerüst der App an.

Zeigt eine lokale Vorschau.

Übernimmt Änderungen beim Entwickeln.

npm installiert die Werkzeuge. Vite hilft uns beim Start und beim Testen.

localhost: eure eigene Vorschau

http://localhost:5173

localhost

Dieser Rechner.

5173

Der Anschluss für den laufenden Vorschau-Server.

Claude-Anfragen können trotzdem über das Internet laufen.

Die Adresse im Terminal ist maßgeblich. Der Port kann auch anders lauten.

Git

Die Geschichte eures Projekts

Git hält ausgewählte Änderungen als Versionen fest.

Ein Commit ist ein benannter Projektstand:
„Spiel startet und zählt Punkte“

Vor einem größeren Experiment sichern wir einen funktionierenden Stand.

Speichern, Git und GitHub

Aktion oder Werkzeug	Was dabei passiert
Datei speichern	Der aktuelle Inhalt landet auf der Festplatte.
Git-Commit	Ein ausgewählter Stand kommt in die Versionsgeschichte.
GitHub / GitLab	Dort können Teams Git-Projekte gemeinsam ablegen.

Für unser lokales Spiel brauchen wir keinen neuen GitHub-Account.

Unser erster Start

Im Projektordner

claude

Für die Vorschau

npm run dev

Claude arbeitet an den Dateien.

Ein zweites Terminal hält die Vorschau am Laufen.

Das erste Anlegen machen wir gemeinsam.

Ein verständlicher Auftrag an Claude

Baue ein kleines Spiel „Schraubenfang“.

Ich bewege einen Auffangkorb mit den Pfeiltasten.

Von oben fallen Schrauben. Jeder Treffer gibt einen Punkt.

Eine Runde dauert 30 Sekunden. Danach kann ich neu starten.

Nutze das vorhandene React-Projekt und einfache Formen.

Das Spiel soll ohne Anmeldung und externe Daten funktionieren.

Danach prüfen wir im Browser, ob genau dieses Verhalten funktioniert.

Wenn etwas hakt

**„Das sollte passieren.
Das passiert tatsächlich.“**

Dazu: Fehlermeldung oder Screenshot.

Dann eine Änderung ausprobieren und erneut testen.

„Warum?“ und „Erklär mir den nächsten Schritt“ sind gute Rückfragen.

30 Minuten für euer erstes Spiel

- 5 Minuten** Eine Spielidee und einfache Regeln.
- 15 Minuten** Die erste spielbare Version.
- 5 Minuten** Selbst testen und eine Sache verbessern.
- 5 Minuten** Gegenseitig ausprobieren.

Jeder baut zunächst selbst. Manuel und Michael helfen.



Eure Aufgabe als erster Ausschnitt

Eine konkrete Eingabe

Ein Beispieltranskript.

Eine kleine Tabelle.

Ein freigegebenes Beispieldokument.

Ein prüfbares Ergebnis

Ein brauchbarer Story-Entwurf.

Eine Liste auffälliger Werte.

Eine Anleitung mit klaren Schritten.

Eine funktionierende Aufgabe reicht für den Anfang.

Eine Vorlage als Startpunkt

- 1 Kit herunterladen** Ein ZIP auswählen und vollständig entpacken.
- 2 Ordner öffnen** Claude Code im Ordner mit der README starten.
- 3 Auftrag einfügen** „Lies CLAUDE.md und README.json. Baue die Grundaufgabe.“

Die README erklärt Start, Aufgabe und Prüfung Schritt für Schritt.

Drei Starter für E-Commerce

Eigenes Tool	Die Bauaufgabe
Produktdaten-Check	Fehlende Pflichtangaben und doppelte Artikelnummern in einer CSV finden.
Produktvergleich	Zwei oder drei Produkte auswählen und Unterschiede nebeneinander zeigen.
Shop-Reporting	Viele Besuche und wenige Bestellungen erkennen; Ergebnisse filtern und exportieren.

manuel-fuss.de/festool

Jedes Kit läuft eigenständig mit erfundenen Beispieldaten.

Ein erster Ablauf in Copilot Studio

- 1 Eingabe** Eine Beispielanfrage als Text.
- 2 Verarbeitung** Anliegen erkennen und fehlende Angaben markieren.
- 3 Ergebnis** Einen Antwortentwurf erzeugen.
- 4 Prüfung** Ein Mensch liest und entscheidet über die Verwendung.

Eine echte Postfachanbindung kommt erst mit passendem Zugriff und klaren Regeln dazu.

Zusatzaufgabe: ein Produktberater

Passende Produkte aus drei Antworten

Anwendung, Stromversorgung und Budget abfragen.
Treffer mit klaren Regeln auswählen und begründen.
Auch fehlende Angaben und keine Treffer berücksichtigen.

Die Aufgabe und Prüffälle stehen im Produktvergleich unter BONUS.md und BONUS.json.

Ein kurzer Blick auf euren Zwischenstand

Was habt ihr gebaut?

Was funktioniert schon?

Ein kurzer Versuch am Bildschirm genügt.

Wer möchte, zeigt auch die Stelle, an der es gerade hakt.

Die Vorstellungsrunde ist freiwillig.

Unterlagen und nächster Schritt

Projekt behalten

Ordner und funktionierenden Git-Stand sichern.

Start festhalten

Die KI um eine kurze README mit Startschritten bitten.

Nächsten Schritt notieren

Was funktioniert, was fehlt und was ihr prüfen wollt.



PDF & Starter-Kits

manuel-fuss.de/festool